

# Tree Traversal & Binary Trees

Shahriar Ivan

Lecturer, Department of CSE, IUT

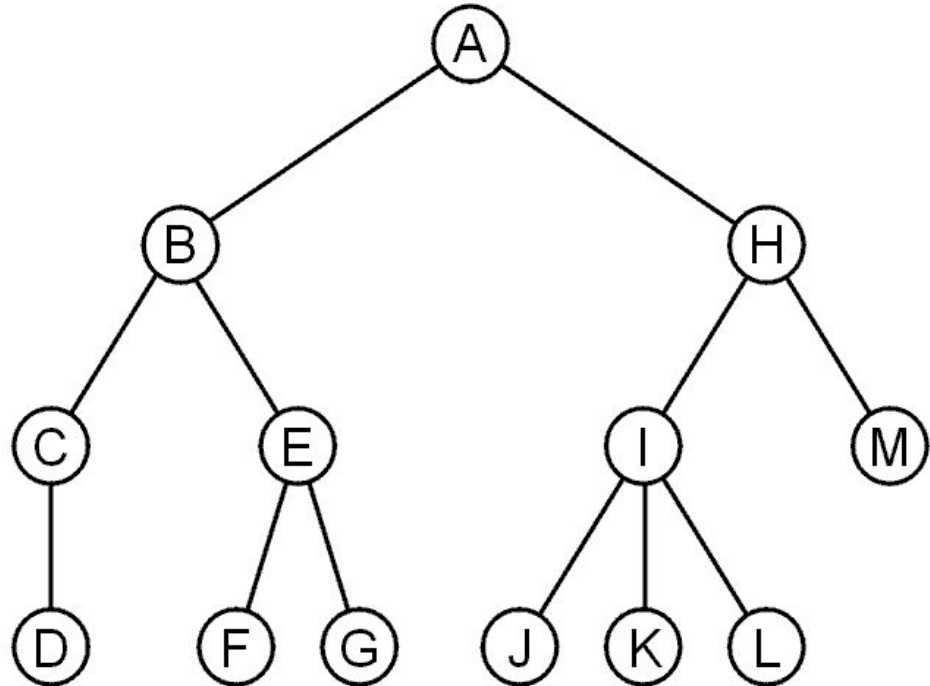


# Abstract Trees

An abstract tree (or abstract hierarchy) does not restrict the number of nodes

In this tree, the degrees vary:

| Degree | Nodes               |
|--------|---------------------|
| 0      | D, F, G, J, K, L, M |
| 1      | C                   |
| 2      | A, B, E, H          |
| 3      | I                   |



# Abstract Trees - Design

We implement an abstract tree or hierarchy by using a class that:

- Stores a value
- Stores the children in a linked-list

# Implementation

The class definition would be:

```
template <typename Type>
class Simple_tree {
    private:
        Type node_value;
        Simple_tree *parent_node;
        Single_list<Simple_tree *> children;

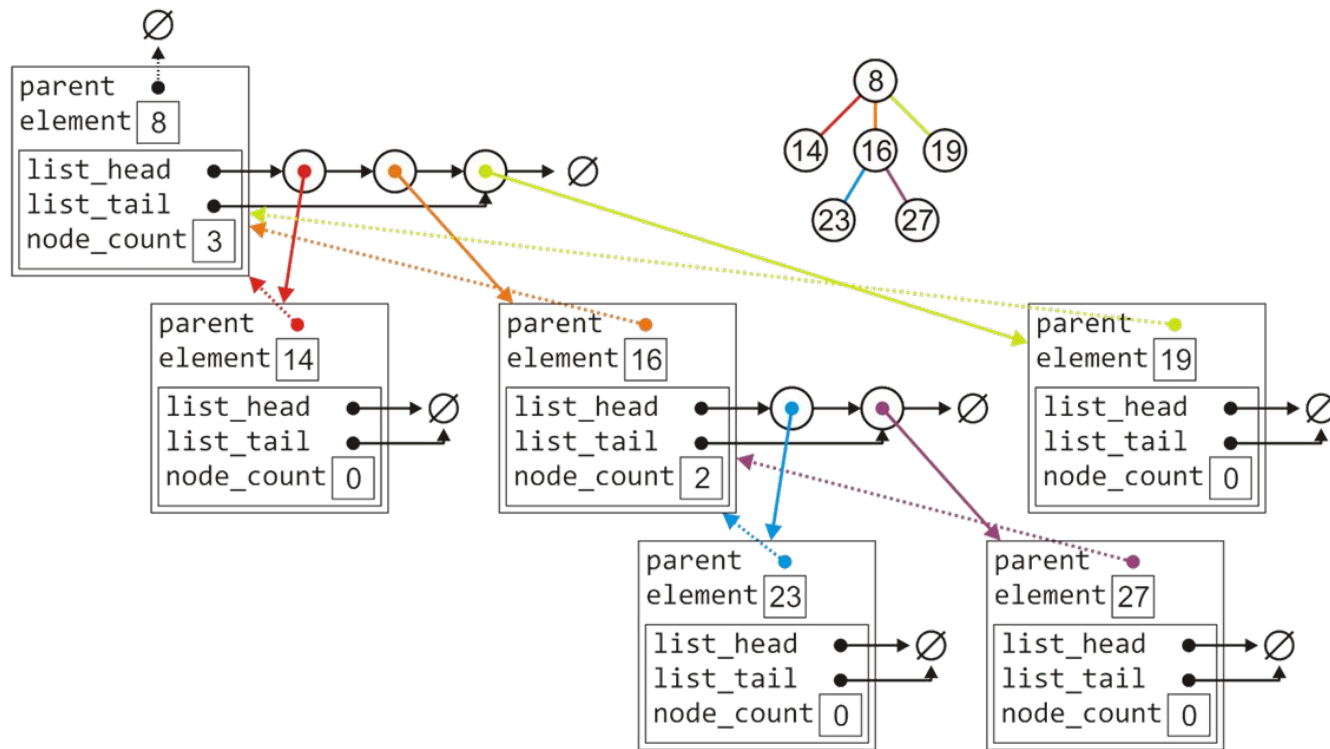
    public:
        Simple_tree( Type const & = Type(), Simple_tree * = nullptr );

        Type value() const;
        Simple_tree *parent() const;
        int degree() const;
        bool is_root() const;
        bool is_leaf() const;
        Simple_tree *child( int n ) const;
        int height() const;

        void insert( Type const & );
        void attach( Simple_tree * );
        void detach();
};
```

# Implementation

The tree with six nodes would be stored as follows:



# Implementation

Much of the functionality is similar to that of the `Single_list` class:

```
template <typename Type>
Simple_tree<Type>::Simple_tree( Type const &obj, Simple_tree *p ):
node_value( obj ),
parent_node( p ) {
    // Empty constructor
}

template <typename Type>
Type Simple_tree<Type>::value() const {
    return node_value;
}

template <typename Type>
Simple_tree<Type> *Simple_tree<Type>::parent() const {
    return parent_node;
}
```

# Implementation

Much of the functionality is similar to that of the Single\_list class:

```
template <typename Type>
bool Simple_tree<Type>::is_root() const {
    return ( parent() == nullptr );
}
```

```
template <typename Type>
int Simple_tree<Type>::degree() const {
    return children.size();
}
```

```
template <typename Type>
bool Simple_tree<Type>::is_leaf() const {
    return ( degree() == 0 );
}
```

# Implementation

Accessing the  $n$ th child requires a for loop ( $\Theta(n)$ ):

```
template <typename Type>
Simple_tree<Type> *Simple_tree<Type>::child( int n ) const {
    if ( n < 0 || n >= degree() ) {
        return nullptr;
    }

    auto *child = children.head();

    // Skip the first n - 1 children
    for ( int i = 1; i < n; ++i ) {
        child = child->next();
    }

    return child->value();
}
```



# Implementation

Attaching a new object to become a child is similar to a linked list:

```
template <typename Type>
void Simple_tree<Type>::attach( Type const &obj ) {
    children.push_back( new Simple_tree( obj, this ) );
}
```

# Implementation

To detach a tree from its parent:

- If it is already a root, do nothing
- Otherwise, erase this object from the parent's list of children and set the parent pointer to zero

```
template <typename Type>
void Simple_tree<Type>::detach() {
    if ( is_root() ) {
        return;
    }

    parent()->children.erase( this );
    parent_node = nullptr;
}
```

# Implementation

Attaching an entirely new tree as a sub-tree, however, first requires us to check if the tree is not already a sub-tree of another node:

- If so, we must detach it first and only then can we add it

```
template <typename Type>
void Simple_tree<Type>::attach( Simple_tree<Type> *tree ) {
    if ( !tree->is_root() ) {
        tree->detach();
    }

    tree->parent_node = this;
    children.push_back( tree );
}
```

# Implementation

Suppose we want to find the size of a tree:

- An empty tree has size 0, a tree with no children has size 1
- Otherwise, the size is one plus the size of all the children

```
template <typename Type>
int Simple_tree<Type>::size() const {
    if ( this == nullptr ) {
        return 0;
    }

    int tree_size = 1;

    for ( auto *child = children.begin(); child != children.end(); child = child->next() ) {
        tree_size += child->value()->size();
    }

    return tree_size ;
}
```

# Implementation

Suppose we want to find the height of a tree:

- An empty tree has height  $-1$  and a tree with no children is height  $0$
- Otherwise, the height is one plus the maximum height of any sub tree

```
#include <algorithm>
```

```
template <typename Type>
```

```
int Simple_tree<Type>::height() const {
```

```
    if ( this == nullptr ) {
```

```
        return -1;
```

```
    }
```

```
    int tree_height = 0;
```

```
    for ( auto *child = children.begin(); child != children.end(); child = child->next() ) {
```

```
        tree_height = std::max( h, 1 + child->value()->height() );
```

```
    }
```

```
    return tree_height;
```

```
}
```

# Tree Traversal

We have already seen one traversal:

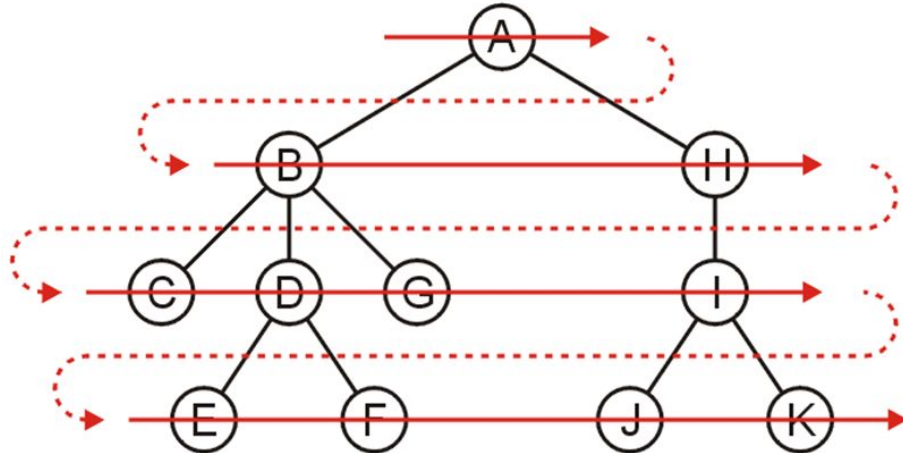
- The breadth-first traversal visits all nodes at depth  $k$  before proceeding onto depth  $k + 1$
- Easy to implement using a queue

Another approach is to visit always go as deep as possible before visiting other siblings:  
depth-first traversals

# Breadth-First Traversal

Breadth-first traversals visit all nodes at a given depth

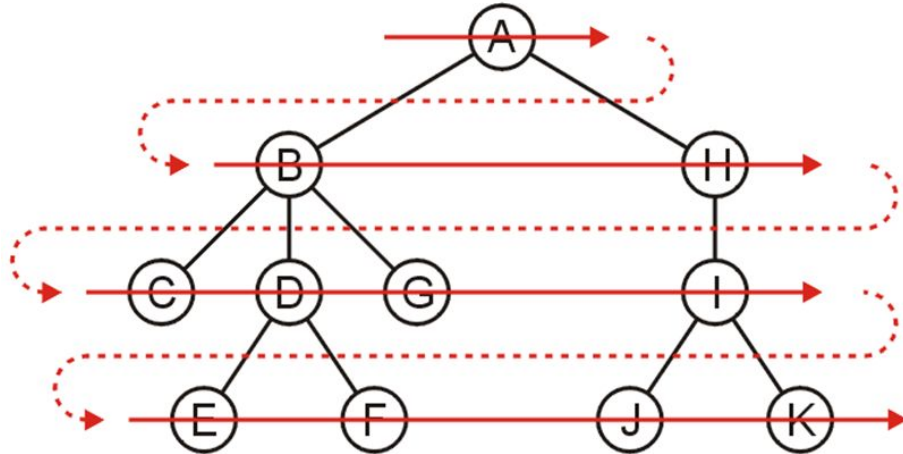
- Can be implemented using a queue
- Run time is  $\Theta(n)$
- Memory is potentially expensive: maximum nodes at a given depth
- Order: A B H C D G I E F J K



# Breadth-First Traversal

The implementation was already discussed:

- Create a queue and push the root node onto the queue
- While the queue is not empty:
  - Push all of its children of the front node onto the queue
  - Pop the front node



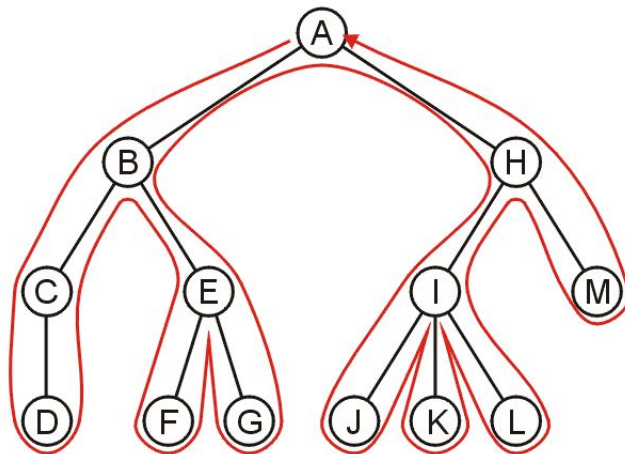


# Backtracking

To discuss depth-first traversals, we will define a backtracking algorithm for stepping through a tree:

- At any node, we proceed to the first child that has not yet been visited
- Or, if we have visited all the children (of which a leaf node is a special case), we backtrack to the parent and repeat this decision making process

We end once all the children of the root are visited

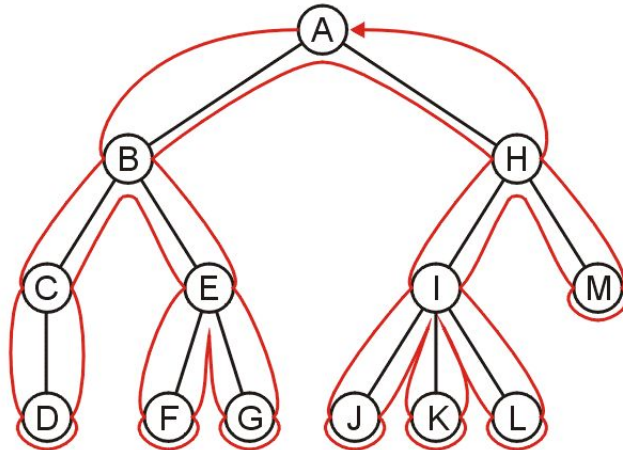


# Depth-first Traversal

We define such a path as a depth-first traversal

We note that each node could be visited twice in such a scheme

- The first time the node is approached (before any children)
- The last time it is approached (after all children)



# DFS Implementation

Depth-first traversals can be implemented with recursion:

```
template <typename Type>
void Simple_tree<Type>::depth_first_traversal() const {
    // Perform pre-visit operations on the value
    std::cout << "<" << node_value << ">";

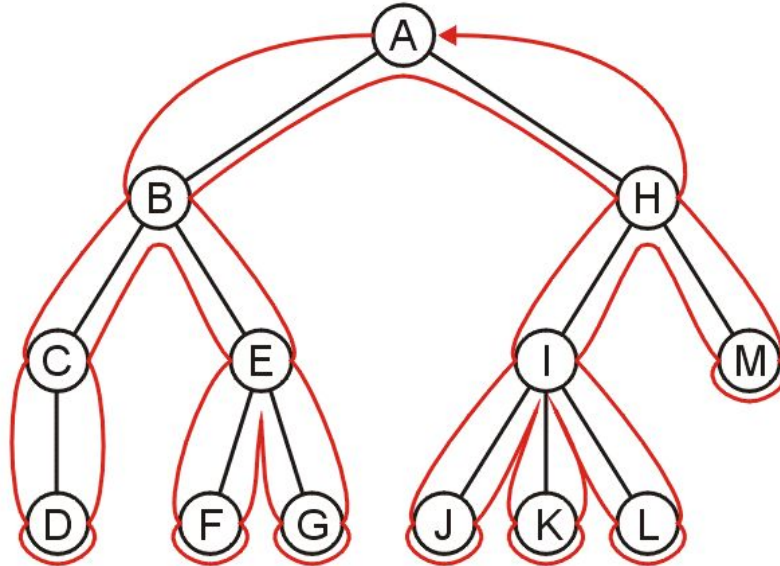
    // Perform a depth-first traversal on each of the children
    for ( auto *child = children.head(); child != children.end();
          child = ptr->next() ) {
        child->value()->depth_first_traversal();
    }

    // Perform post-visit operations on the value
    std::cout << "</" << node_value << ">";
}
```

# DFS Implementation

Performed on this tree, the output would be

```
<A><B><C><D></D></C><E><F></F><G></G></E></B><H><I>  
<J></J><K></K><L></L></I><M></M></H></A>
```



# DFS Implementation using Stack

Alternatively, we can use a stack:

- Create a stack and push the root node onto the stack
- While the stack is not empty:
  - Pop the top node
  - Push all of the children of that node to the top of the stack in reverse order
- Run time is  $\Theta(n)$
- The objects on the stack are all unvisited siblings from the root to the current node
- If each node has a maximum of two children, the memory required is  $\Theta(h)$ : the height of the tree

With the recursive implementation, the memory is  $\Theta(h)$ : recursion just hides the memory

# Binary Tree

The arbitrary number of children in general trees is often unnecessary—many real-life trees are restricted to two branches

- Expression trees using binary operators
- An ancestral tree of an individual, parents, grandparents, etc.
- Phylogenetic trees
- Lossless encoding algorithms

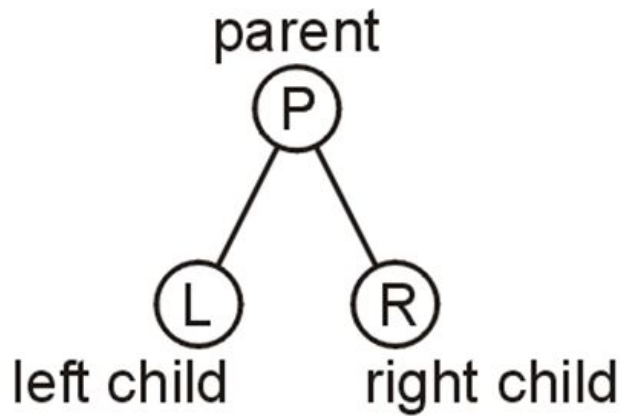
There are also issues with general trees:

- There is no natural order between a node and its children

# Binary Tree

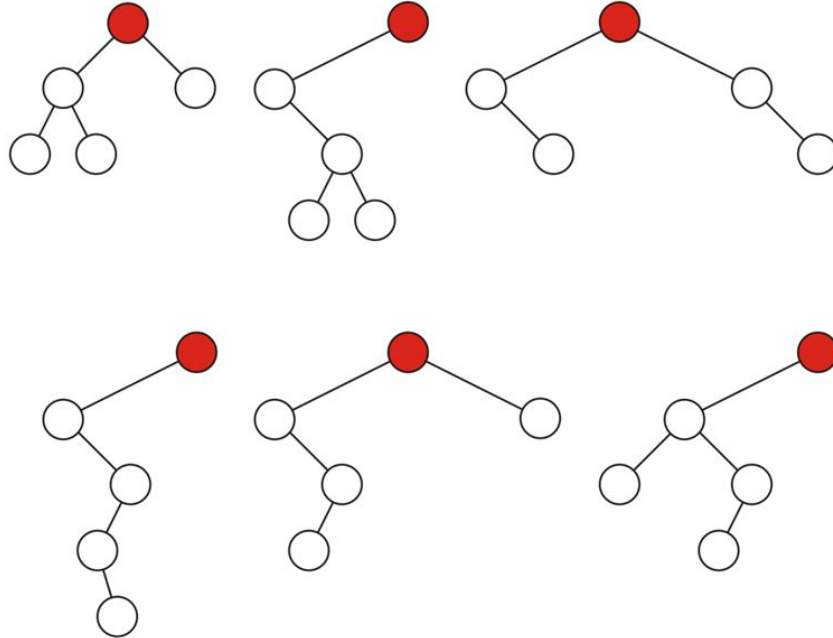
A binary tree is a restriction where each node has exactly two children:

- Each child is either empty or another binary tree
- This restriction allows us to label the children as left and right subtrees



# Binary Tree

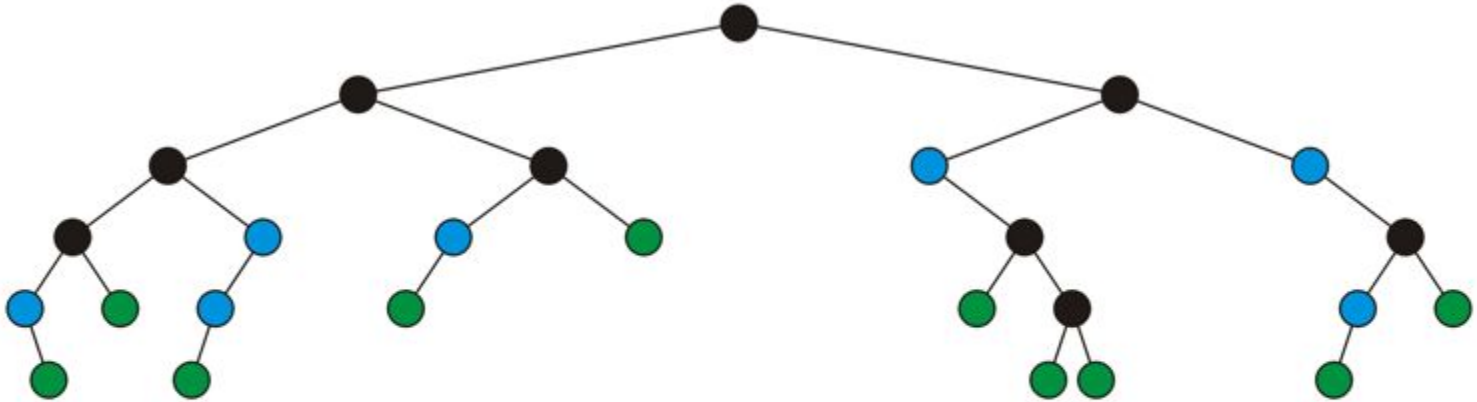
Sample variations on binary trees with five nodes:





# Binary Tree

A full node is a node where both the left and right sub-trees are non-empty trees



Legend:

full nodes ●

neither ●

leaf nodes ●

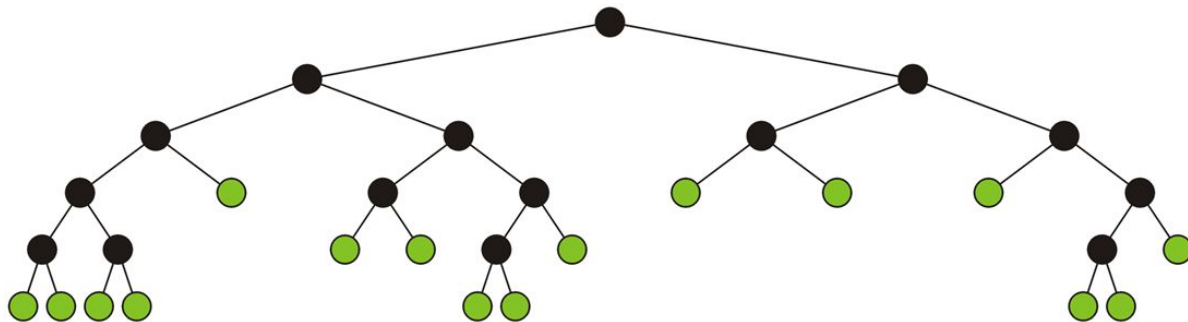
# Binary Tree

A full binary tree is where each node is:

- A full node, or
- A leaf node

These have applications in

- Expression trees
- Huffman encoding



# Binary Node class

The binary node class is similar to the single node class:

```
#include <algorithm>

template <typename Type>
class Binary node {
protected:
    Type node value;
    Binary node *p left tree;
    Binary node *p right tree;

public:
    Binary node( Type const & );

    Type value() const;
    Binary node *left() const;
    Binary node *right() const;

    bool is leaf() const;
    int size() const;
    int height() const;
    void clear();
}
```

# Binary Node class

We will usually only construct new leaf nodes

```
template <typename Type>
Binary_node<Type>::Binary_node( Type const &obj ):
node_value( obj ),
p_left_tree( nullptr ),
p_right_tree( nullptr ) {
    // Empty constructor
}
```

# Binary Node class

The accessors are similar to that of Single\_list

```
template <typename Type>
Type Binary_node<Type>::value() const {
    return node_value;
}
```

```
template <typename Type>
Binary_node<Type> *Binary_node<Type>::left() const {
    return p_left_tree;
}
```

```
template <typename Type>
Binary_node<Type> *Binary_node<Type>::right() const {
    return p_right_tree;
}
```

# Binary Node class

Much of the basic functionality is very similar to Simple\_tree

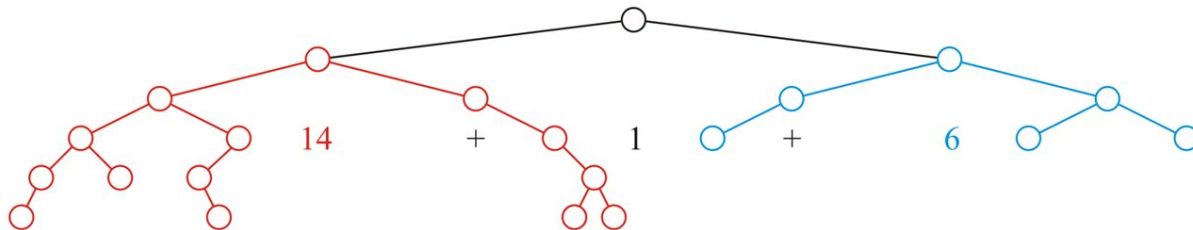
```
template <typename Type>
bool Binary_node<Type>::is_leaf() const {
    return (left() == nullptr) && (right() == nullptr);
}
```

# Binary Node class

The recursive size function runs in  $\Theta(n)$  time and  $\Theta(h)$  memory

- These can be implemented to run in  $\Theta(1)$

```
template <typename Type>
int Binary_node<Type>::size() const {
    if ( left() == nullptr ) {
        return ( right() == nullptr ) ? 1 : 1 + right()->size();
    } else {
        return ( right() == nullptr ) ?
            1 + left()->size() :
            1 + left()->size() + right()->size();
    }
}
```



# Binary Node class

The recursive height function also runs in  $\Theta(n)$  time and  $\Theta(h)$  memory

- Can also be implemented to run in  $\Theta(1)$

```
int Binary_node<Type>::height() const {  
    if ( left() == nullptr ) {  
        return ( right() == nullptr ) ? 0 : 1 + right()->height();  
    } else {  
        return ( right() == nullptr ) ?  
            1 + left()->height() :  
            1 + left()->height() + right()->height();  
    }  
}
```



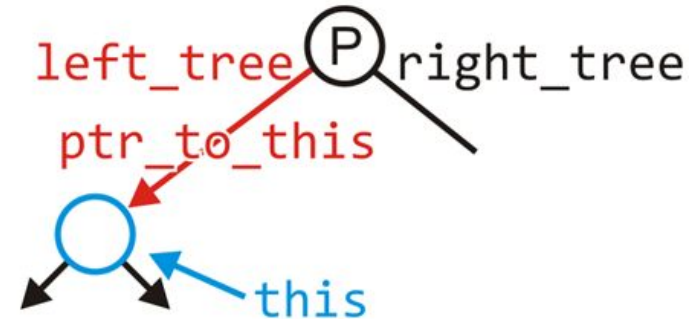
# Binary Node class

Removing all the nodes in a tree is similarly recursive:

```
template <typename Type>
void Binary_node<Type>::clear( Binary_node *&p_to_this ) {
    if ( left() != nullptr ) {
        left()->clear( p_left_tree );
    }

    if ( right() != nullptr ) {
        right()->clear( p_right_tree );
    }

    delete this;
    p_to_this = nullptr;
}
```



# Run Times

Recall that with linked lists and arrays, some operations would run in  $\Theta(n)$  time

The run times of operations on binary trees, we will see, depends on the height of the tree

We will see that:

- The worst is clearly  $\Theta(n)$
- Under average conditions, the height is  $\Theta(\sqrt{n})$
- The best case is  $\Theta(\lg(n))$